

ETL Observability Platform

End-to-end architecture (simple guide)

We do **not** rebuild ETL. We **watch** databases and ETL tools, **normalize** their metadata and logs, and **store** them in **one Metadata database** so we can see, for each pipeline: source, ETL tool, target, and errors.

Open ETL tools (dbt / Airflow) move the data.
Our platform stores the story of that data movement.

1. Big picture architecture

Figure 1 — Big picture architecture

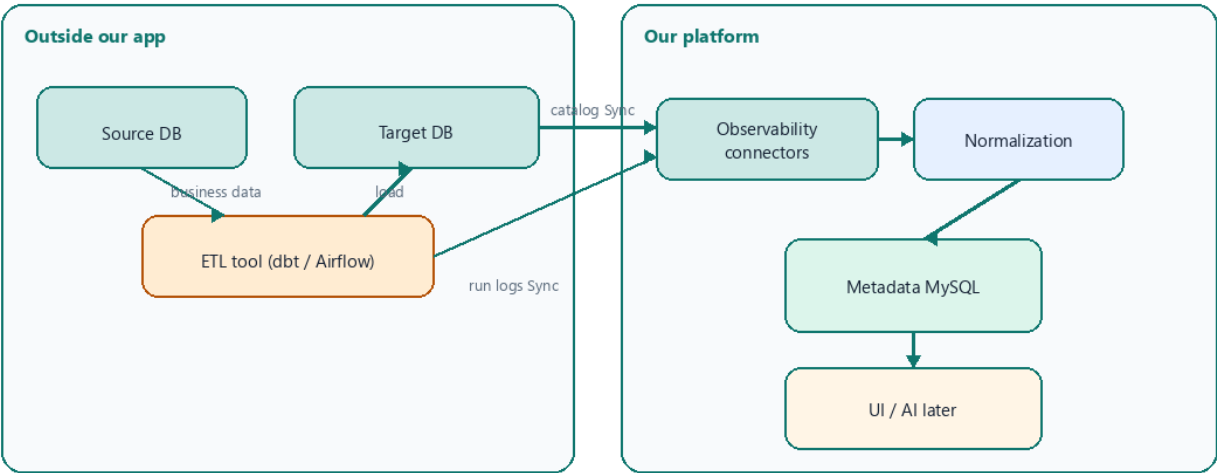


Figure 1 — Outside tools sync into our Metadata DB

- Source DB / Target DB — real warehouses (e.g. Snowflake). Business data lives there.
- ETL tool — dbt or Airflow. It already has extract/load connectors. It runs the pipeline.
- Our connectors — thin readers. They Sync metadata and logs into our DB. They do not move business rows.
- Normalization — turns each tool's messy JSON into one standard shape.
- Metadata MySQL — one place for pipelines, tables, runs, errors, links.
- UI / AI — later read only Metadata (not live Snowflake at chat time).

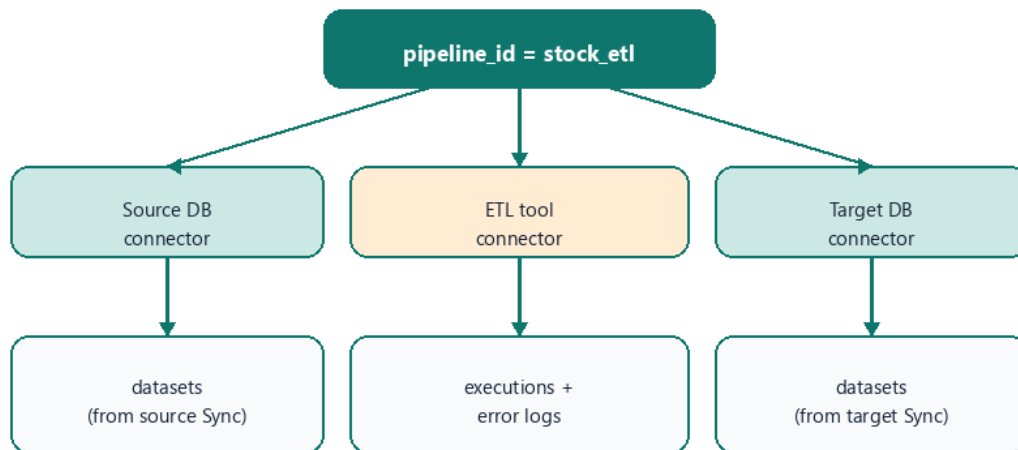
Two meanings of “connector”

Kind	Who owns it	Job
ETL extract/load connectors	dbt / Airflow	Move data source → target
Our observability connectors	Our platform	Collect metadata + logs into Metadata DB

2. How a pipeline is recognized (pipeline_id)

You create one **pipeline** and attach three things: Source DB connector, ETL tool connector, Target DB connector. Usual MVP: **1 ETL + 2 DB roles**.

Figure 2 — Pipeline ID attachments



Everything is found again using the same pipeline_id.

Figure 2 — Attachments under one pipeline_id

- pipeline_id is the folder name for one data flow.
- After Sync: show source tables, target tables, last failures for that id.
- Linking is stored in etl_pipeline_io. Without it, Snowflake and dbt stay separate piles.

3. How connectors work

Figure 3 — Connector day-to-day flow



Test = login works. Sync = pull metadata/logs into Metadata MySQL.

Figure 3 — Create → Test → Sync → Store

Connector	Pulls from	Main result in our DB
Snowflake	INFORMATION_SCHEMA	Tables, row_count, last_updated
dbt Cloud	Runs API / artifacts	Job status, error_message
Airflow (later)	REST / metadata DB	DAG/task runs, errors
MySQL (later)	information_schema	Tables like Snowflake

- Secrets never go in the database — only env var names.
- Assistants never call Snowflake/dbt directly; they read Metadata.

4. Two transforms (do not confuse them)

Transform	Where	What it does
Business ETL transform	Inside dbt/Airflow	Clean/join business rows
Metadata transform	Our Normalization	Raw logs/catalog → structured Metadata tables

R&D for us = better mapping rules and source/target linking — **not** replacing dbt SQL.

5. What we store (clear tables & columns)

All tables live in one MySQL database (e.g. **metadata**). Prefix: **etl_**.

A. Connections — etl_connector_instances

Column	Meaning
tenant_id	Customer/space (e.g. demo)
instance_id	Unique connection id
tool_id	snowflake / dbt / airflow
name	Display name
config	Non-secret settings (account, database, ...)
secrets_ref	Env var names only
status	created / ready / synced / error
last_sync_at	Last successful Sync
last_error	Last error text

B. Pipeline identity — etl_pipelines

Column	Meaning
pipeline_id	Our id (e.g. stock_etl)
name	Human name
source_tool	Main ETL tool (dbt, ...)
status	Latest known status

etl_pipeline_io (source ↔ target)

Column	Meaning
pipeline_id	Which pipeline
upstream_dataset_id	Source table id
downstream_dataset_id	Target table id
source_tool	Tool that declared the link

C. From databases — etl_datasets

Column	Meaning
dataset_id	Full name e.g. ANALYTICS_DB.RAW.STOCK_DATA_RAW
name	Short table name
database_name	Database
schema_name	Schema
platform	snowflake / mysql
row_count	Approx rows (for volume later)
last_updated_at	Last change time (for freshness later)

etl_dataset_columns (later — schema diff)

Column	Meaning
dataset_id	Parent table
column_name	Column
data_type	Type (VARCHAR, NUMBER, ...)
is_nullable	Null allowed?

D. From ETL tools — etl_executions (logs)

Column	Meaning
execution_id	Run id from the tool
pipeline_id	Which pipeline
task_id	Step/model if any
source_tool	dbt / airflow
status	succeeded / failed / running
started_at / finished_at	Run times
error_message	The log / error text
attempt	Retry number

MVP focus: connections + pipelines + pipeline_io + datasets + executions (with error_message).

6. Example: one real pipeline

pipeline_id: stock_etl

Source: ANALYTICS_DB.RAW.STOCK_DATA_RAW

ETL: dbt Cloud (runs in etl_executions)

Target: ANALYTICS_DB.STAGING_STAGING.STG_STOCK_DATA

- Sync Snowflake → rows in etl_datasets
- Sync dbt → rows in etl_executions (status + error_message)
- Link → row in etl_pipeline_io
- Then answer: failed runs? error text? source/target tables?

7. Freshness (later — not MVP UI)

Freshness = “is the **table** updated recently enough?” using last_updated_at vs a time rule. Use it when a job looks green but the target is still old. Failed pipeline counts come from **etl_executions**, not freshness.

8. What we can do using this Metadata DB

Figure 4 — What we can do with Metadata



AI and UI read only Metadata — not live Snowflake/dbt at chat time.

Figure 4 — Downstream uses of Metadata

1. List pipelines and show running / failed counts.
2. Open a pipeline and see source DB, ETL tool, target DB.
3. Show error logs from error_message without opening dbt Cloud.
4. List tables discovered from Snowflake Sync.
5. Filter everything by pipeline_id.
6. Feed one AI assistant with tools that only query Metadata.
7. Later: reports (BIRT / Superset); freshness/volume; lineage UI.
8. Later: schema diff when column types change between Syncs.

9. Point-wise summary (whole story)

1. ETL tools run pipelines; we observe them.
2. Each tool type has an observability connector (Snowflake, dbt, ...).
3. Sync pulls metadata/logs; Normalization standardizes them.
4. Everything lands in one MySQL Metadata DB.
5. A pipeline_id attaches source DB + ETL + target DB.
6. DB connectors fill datasets; ETL connectors fill executions (logs).
7. pipeline_io links source table → target table for that pipeline.
8. MVP UI: fails, errors, attachments — not freshness-first.
9. AI and reports sit on Metadata, not on live tools.
10. We do not rebuild extract/load; we store the story so people and AI can understand it.

Document source: docs/fresh-start/09-e2e-architecture.md